

게임 프로젝트 관련 배운 내용

7-12

정명훈

목표: 서버에 접속하는 python client 제작

1.접속 시 어떤 일이 일어나지?

=> 소켓 관련된 API 호출 => connect / send API 호출

(IDA Imports 탭 활용 / ProcessMonitor 활용)

2.서버에 어떤 데이터를 보내는 지?

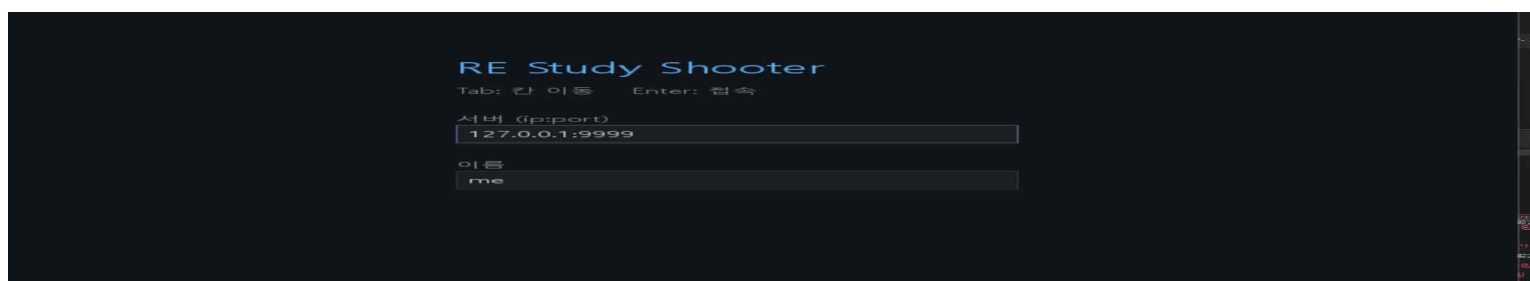
=> GGWx01W0x1<길이><닉네임>

python client 작성하면서 파악하기

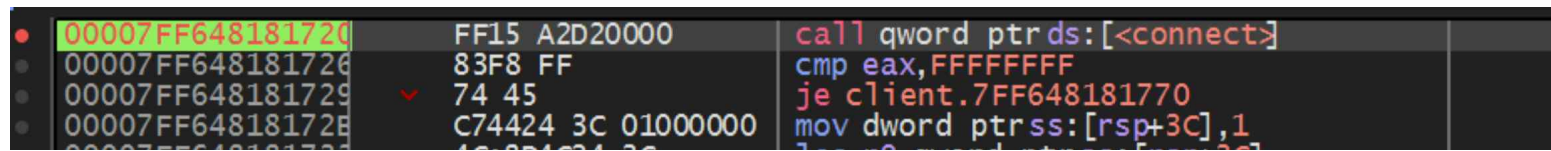
실제 진행

1. process monitor를 활용한 소켓 관련 api호출

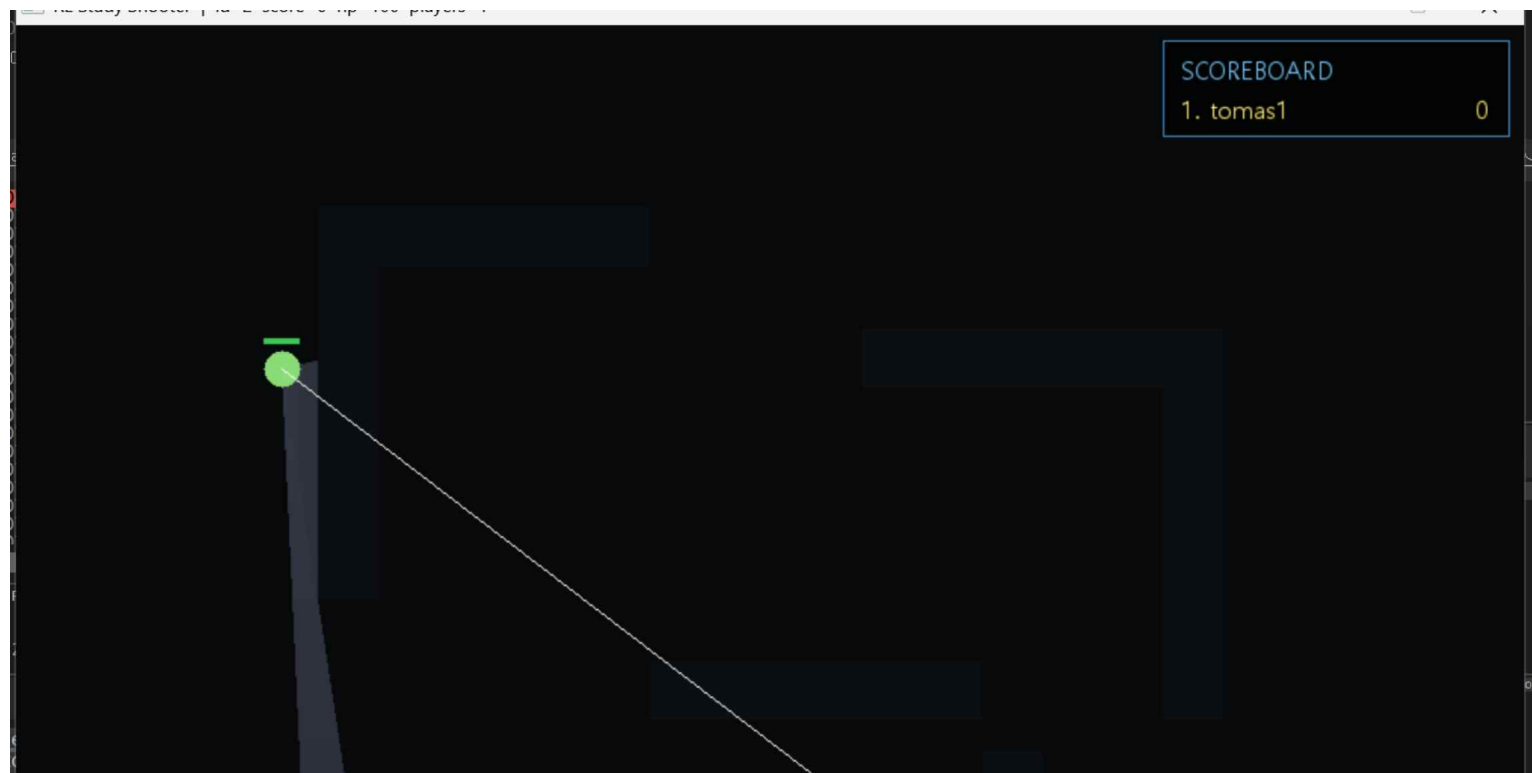
먼저 process monitor를 깔고 그 다음에 server.exe를 실행한다. 그 다음에 xdbg64에 client.exe를 올리고 실행한다.



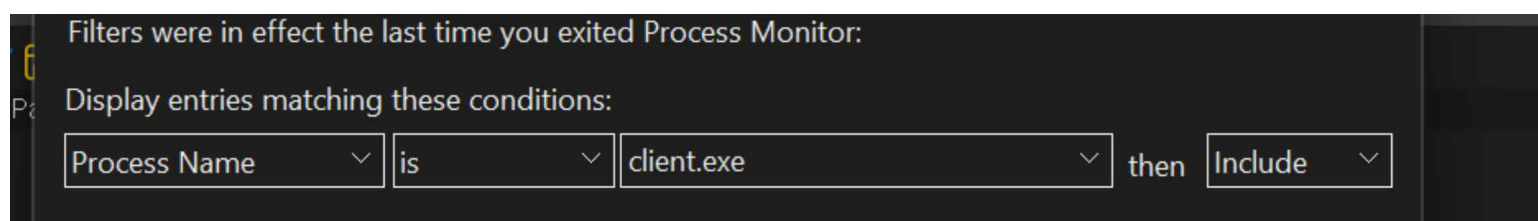
그럼 위의 스크린샷처럼 화면이 뜨는데 그 지점에 xdbg부분에 중단점을 한 번 걸고 그 다음에 이름을 입력하고 엔터를 치고 xdbg화면을 보면



이렇게 뜨고 이를 통해 접속시 connect api를 호출한다는 걸 알 수 있다 connect부분에 중단점을 선택하고 다시client.exe파일을 보면



이와 같이 wasd로 움직이는 걸 확인할 수 있다. 그 다음에는 process monitor를 이용해서 정확히 어떤 데이터를 보내는지 확인해 보자 다시 xdbg를 재시작하고 process monitor를 실행해서



이렇게 설정을 잡고 add를 누르고 ok를 누르면 관련된 것들이 엄청 뜨기 시작한다

여기서 레지스트리 파일과 readfile은 별로 관심이 없기 때문에



여기서 선택해제를 해서 안 보이게 하면

Time ...	Process Name	PID	Operation	Path	Result	Detail
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\directxdata...	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\ResourcePol...	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\ResourcePol...	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\DriverStoreW...	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\DriverStoreW...	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	Thread Create		SUCCESS	Thread ID: 13800
오후 4:...	client.exe	10624	Thread Create		SUCCESS	Thread ID: 7900
오후 4:...	client.exe	10624	Thread Create		SUCCESS	Thread ID: 27060
오후 4:...	client.exe	10624	Thread Create		SUCCESS	Thread ID: 21308
오후 4:...	client.exe	10624	Thread Create		SUCCESS	Thread ID: 6108
오후 4:...	client.exe	10624	Thread Create		SUCCESS	Thread ID: 11312
오후 4:...	client.exe	10624	Thread Create		SUCCESS	Thread ID: 17192
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\CoreMessag...	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\CoreUIComp...	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\WinTypes.dll	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\clbcatq.dll	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	Load Image	C:\Windows\System32\mswsock.dll	SUCCESS	Image Base: 0x7ff...
오후 4:...	client.exe	10624	TCP Connect	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 0, mss: 65...
오후 4:...	client.exe	10624	TCP Send	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 11, startin...
오후 4:...	client.exe	10624	TCP TCPCopy	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 68, sequen...
오후 4:...	client.exe	10624	TCP Receive	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 68, sequen...
오후 4:...	client.exe	10624	Thread Create		SUCCESS	Thread ID: 24288
오후 4:...	client.exe	10624	TCP TCPCopy	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 24, sequen...
오후 4:...	client.exe	10624	TCP Receive	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 24, sequen...
오후 4:...	client.exe	10624	TCP Send	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 9, startin...
오후 4:...	client.exe	10624	TCP TCPCopy	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 21, sequen...
오후 4:...	client.exe	10624	TCP Receive	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 21, sequen...
오후 4:...	client.exe	10624	TCP Send	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 9, startin...
오후 4:...	client.exe	10624	TCP TCPCopy	LAPTOP-UT2CVUO8:58639 -> LAPTOP...	SUCCESS	Length: 21, sequen...

이렇게 서버 연결시 tcp send와 connect api가 보인다. 여기서 어느 정도 원하는 값을 찾았기 때문에 ctrl + e를 눌러 더 이상의 이벤트 수집을 멈추고 분석을 시작한다. 우선 내가 찾기를 원하는 것은 실제 connect가 이루어지는 부분이기 때문에 그 부분을 누르고 stack에 가서

39	ntoskrnl.exe	segrmpex + 0x9115	0x7ff80196401265	C:\Windows\System32\ntoskrnl.exe
40	ntdll.dll	NtDeviceIoControlFile + 0x14	0x7ffb81580144	C:\Windows\System32\ntdll.dll
41	mswsock.dll	NSPStartup + 0x652a	0x7ffb7d8c4eea	C:\Windows\System32\mswsock.dll
42	mswsock.dll	NSPStartup + 0x570c	0x7ffb7d8c40cc	C:\Windows\System32\mswsock.dll
43	mswsock.dll	NSPStartup + 0x527d	0x7ffb7d8c3c3d	C:\Windows\System32\mswsock.dll
44	WS2_32.dll	connect + 0x132	0x7ffb80b9f022	C:\Windows\System32\WS2_32.dll
45	client.exe	client.exe + 0x1726	0x7ff648181726	D:\Downloads\temp_1783924517660.-4871\
46	client.exe	client.exe + 0x362e	0x7ff64818362e	D:\Downloads\temp_1783924517660.-4871\
47	client.exe	client.exe + 0x55fc	0x7ff6481855fc	D:\Downloads\temp_1783924517660.-4871\
48	client.exe	client.exe + 0x10c9	0x7ff6481810c9	D:\Downloads\temp_1783924517660.-4871\
49	client.exe	client.exe + 0x13f6	0x7ff6481813f6	D:\Downloads\temp_1783924517660.-4871\
50	KERNEL32.DLL	BaseThreadInitThunk + 0x17	0x7ffb7f3be957	C:\Windows\System32\KERNEL32.DLL
51	ntdll.dll	RtlUserThreadStart + 0x2c	0x7ffb814cad6c	C:\Windows\System32\ntdll.dll

client.exe가 처음 나오는 부분을 클릭하고 그 부분의 주소로 이동하면 실제 connect호출후 복귀 주소가 나오고 그 위에는

00007FF648181710	48:89F9	mov rcx,rdi
00007FF648181720	FF15 A2D20000	call qword ptr ds:[<connect>]
00007FF648181726	83F8 FF	cmp eax,FFFFFFFF
00007FF648181729	74 45	je client.7FF648181770
00007FF64818172E	C74424 3C 01000000	mov dword ptr ss:[rsp+3C],1
00007FF648181733	4C:8D4C24 3C	lea r9,qword ptr ss:[rsp+3C]
00007FF648181738	41:B8 01000000	mov r8d,1
00007FF64818173E	48:89F9	mov rcx,rdi
00007FF648181741	C74424 20 04000000	mov dword ptr ss:[rsp+20],4
00007FF648181749	BA 06000000	mov edx,6
00007FF64818174F	FF15 9CD20000	call qword ptr ds:[<connect>]

클라이언트가 서버에 TCP 연결을 시도하는 코드 위치가 나와 있다.

참고로 이 과정은 ida의 import부분의 connect부분을 가서 xref로 호출 지점을 찾아서도 파악할 수 있다.


```

1 __int64 __fastcall connect_to_server(PCSTR pszAddrString, u_short a2)
2 {
3     SOCKET v4; // rdi
4     char optval[4]; // [rsp+3Ch] [rbp-3Ch] BYREF
5     struct sockaddr name; // [rsp+40h] [rbp-38h] BYREF
6
7     v4 = socket(2, 1, 0);
8     if ( v4 != -1 )
9     {
10         name.sa_family = 2;
11         *(_QWORD *)&name.sa_data[6] = 0;
12         *(_QWORD *)name.sa_data = htons(a2);
13         if ( inet_pton(2, pszAddrString, &name.sa_data[2]) == 1 && connect(v4, &name, 16) != -1 )
14         {
15             *(_DWORD *)optval = 1;
16             setsockopt(v4, 6, 1, optval, 4);
17             g_sock = v4;
18             return 1;
19         }
20         closesocket(v4);
21     }
22     return 0;
23 }

```

ida로 보면 참고로 이렇게 나오는 걸 확인할 수 있다. 참고로 이 경우에는 ida를 통한 정적분석으로 찾는게 더 쉬운 편이다 따라서 마찬가지로 send함수도 ida를 통해서 찾아보면

```

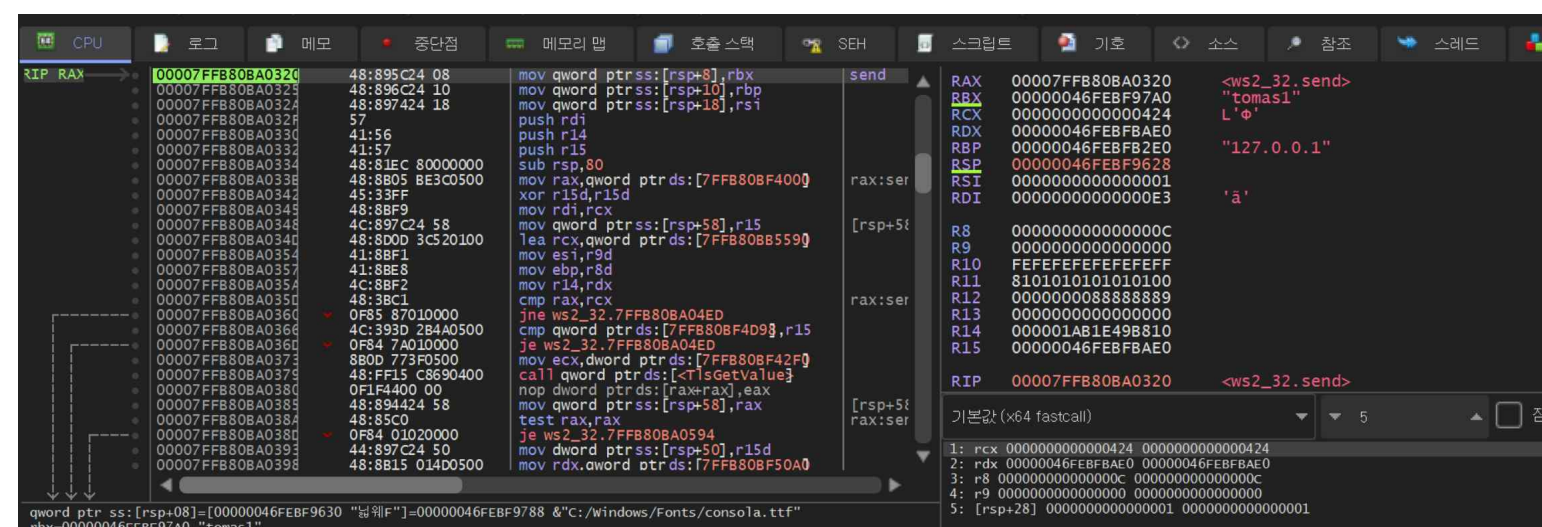
472         *(_WORD *)&Buffer[(unsigned int)v31 + 4] = *(_WORD *)&v281[(unsigned int)v31 + 4];
473     }
474 }
475 send(g_sock, Buffer, v32 + 6, 0);
476 Thread = CreateThread(nullptr, 0, rx_thread, nullptr, 0, nullptr);
477 if ( Thread )
478     CloseHandle(Thread);
479 v34 = fopen_0("client.cfg", "w");
480 v35 = v34;

```

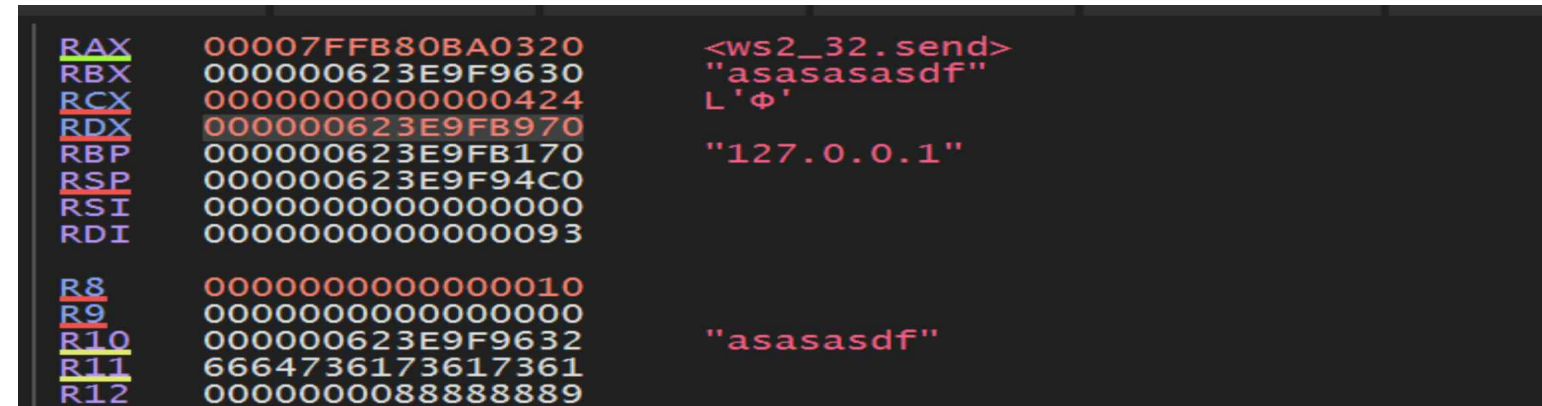
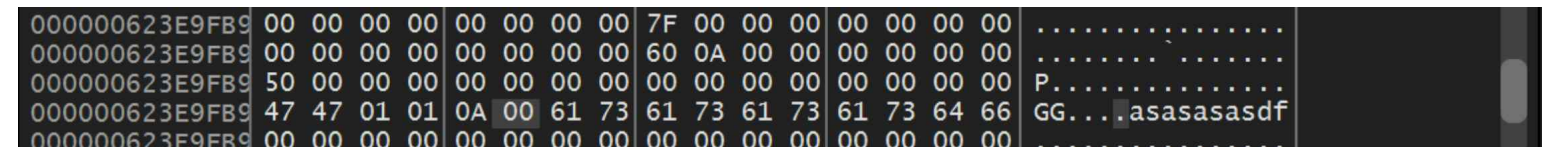
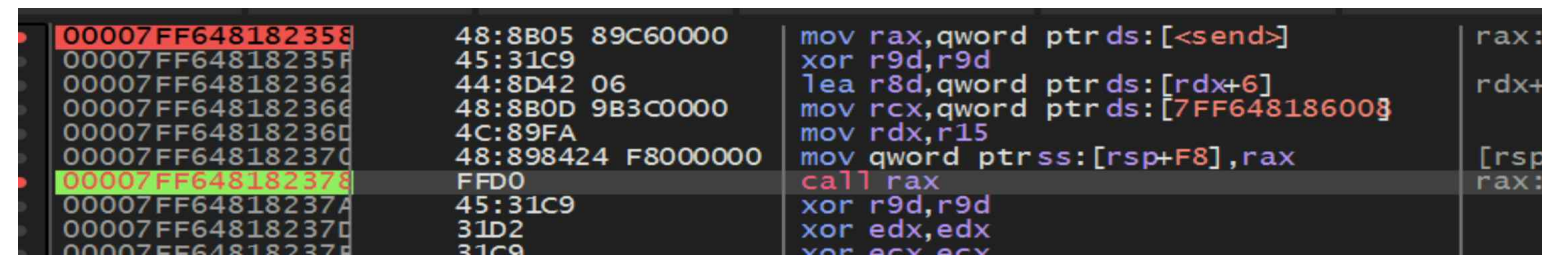
이렇게 나오고 여기 부분을 ida와 xdbg주소를 맞추고 그 위치를 xdbg에서 찾으면

00007FF64818234F	48:85C0	test rax,rax	
00007FF648182352	0F85 201F0000	jne client.7FF648184278	
00007FF648182358	48:8B05 89C60000	mov rax,qword ptr ds:[<send>]	
00007FF64818235F	45:31C9	xor r9d,r9d	
00007FF648182362	44:8D42 06	lea r8d,qword ptr ds:[rdx+6]	
00007FF648182366	48:8B0D 9B3C0000	mov rcx,qword ptr ds:[7FF648186008]	rcx:Nt
00007FF64818236D	4C:89FA	mov rdx,r15	
00007FF648182370	48:898424 F8000000	mov qword ptr ss:[rsp+F8],rax	
00007FF648182378	FFD0	call rax	
00007FF64818237A	45:31C9	xor r9d,r9d	
00007FF64818237D	31D2	xor edx,edx	
00007FF64818237F	31C9	xor ecx,ecx	
00007FF648182381	48:C74424 28 00000000	mov qword ptr ss:[rsp+28],0	
00007FF64818238A	4C:8D05 7FF6FFFF	lea r8,qword ptr ds:[7FF648181A10]	
00007FF648182391	C74424 20 00000000	mov dword ptr ss:[rsp+20],0	
00007FF648182399	FF15 19C20000	call qword ptr ds:[<CreateThread>]	
00007FF64818239F	48:85C0	test rax,rax	
00007FF6481823A2	74 09	je client.7FF6481823AD	
00007FF6481823A4	48:89C1	mov rcx,rax	rcx:Nt
00007FF6481823A7	FF15 03C20000	call qword ptr ds:[<CloseHandle>]	
00007FF6481823AB	48:8D15 4D4D0000	lea rdx,qword ptr ds:[7FF648187101]	
00007FF6481823B4	48:8D0D 624C0000	lea rcx,qword ptr ds:[7FF648187010]	rcx:Nt
00007FF6481823B8	E8 50370000	call <JMP.&fopen>	
00007FF6481823C0	48:89C6	mov rsi,rax	rsi:"m
00007FF6481823C3	48:85C0	test rax,rax	
00007FF6481823C6	74 1F	je client.7FF6481823F7	

이렇게 실제 send 위치를 찾을 수 있다. 그리고 내부의 dll에서 동작을 보기 위해서 call rax안으로 f7을 눌러서 진행하면



send함수의 dll에서 실행이 보이고 옆의 레지스터 값에는 내가 입력한 값들이 보이는 걸 확인할 수 있다. 그 다음에는 실제로 내가 입력한 값이 어떤 형식으로 서버에게 전달이 되는지 확인해 보면



처럼 서버에게

struct Packet {

```
char magic[2];           // "GG"

uint8_t field1;          // 0x01

uint8_t field2;          // 0x01
```

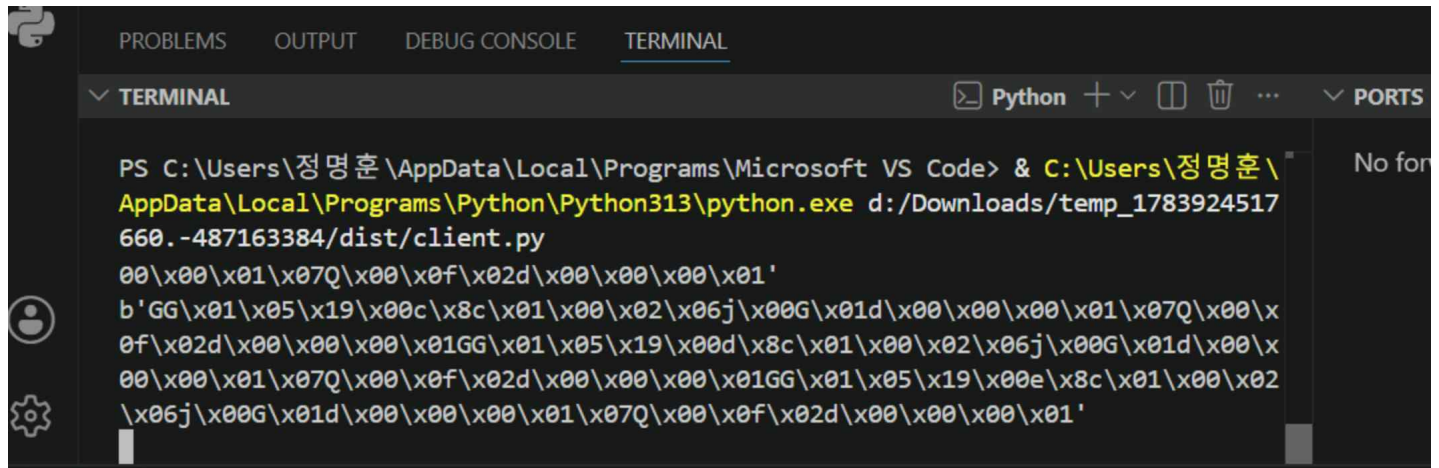
```
uint16_t payload_len;    // 0x000A = 10, little-endian
```

```
char payload[];          // "asasasasdf"
```

```
};
```

이와 같은 형태로 패킷을 보내고 있다는 것을 알 수 있다.

이제 그 다음에는 실제로 서버에 파이썬 파일을 통해 연결 및 데이터 전송을 위해 파이썬 파일 client.py를 작성했고 이는 source폴더에서 확인할 수 있다 이를 작동시키면



```
PS C:\Users\정명훈\AppData\Local\Programs\Microsoft VS Code> & C:\Users\정명훈\AppData\Local\Programs\Python\Python313\python.exe d:/Downloads/temp_1783924517660.-487163384/dist/client.py
00\x00\x01\x07Q\x00\x0f\x02d\x00\x00\x00\x01 '
b'GG\x01\x05\x19\x00c\x8c\x01\x00\x02\x06j\x00G\x01d\x00\x00\x00\x01\x07Q\x00\x0f\x02d\x00\x00\x00\x01GG\x01\x05\x19\x00d\x8c\x01\x00\x02\x06j\x00G\x01d\x00\x00\x00\x01\x07Q\x00\x0f\x02d\x00\x00\x00\x01GG\x01\x05\x19\x00e\x8c\x01\x00\x02\x06j\x00G\x01d\x00\x00\x00\x01\x07Q\x00\x0f\x02d\x00\x00\x00\x01'
```

이렇게 서버와 통신을 통해 받은 데이터를 계속 출력한 모습과



실제로 접속되어 있는 것도 확인할 수 있다. 이와 같이 어떤 서버를 이용하는 프로그램에는 exe파일의 실행뿐만 아니라 이렇게 python코드를 통해서도 서버와 소통하며 프로그램을 간접 실행할 수 있다는 점을 배웠다